Information Concepts and Processing

Concepts of Data

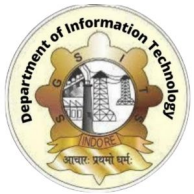
Data

- Data is the **raw material** of information.
- It represents unprocessed facts, symbols, numbers, characters, images, or sounds.
- On its own, data has **no meaning** until it is processed.

👉 Example:

- "98, 87, 76" – These numbers alone are meaningless data.

Concepts of Information



- Information is **processed, organized, and meaningful data**.
- It reduces **uncertainty** and supports **decision-making**.
- Information = Data + Context + Meaning.

👉 Example:

- If we say, "98, 87, and 76 are the marks of a student in Science, Maths, and English,"
  → This becomes information.

Data	Information
Raw facts, unprocessed.	Processed, meaningful, and useful.
No context or interpretation.	Has context and relevance.
Example: 2000, 3000, 4000.	"Employee salaries are 2000, 3000, 4000".

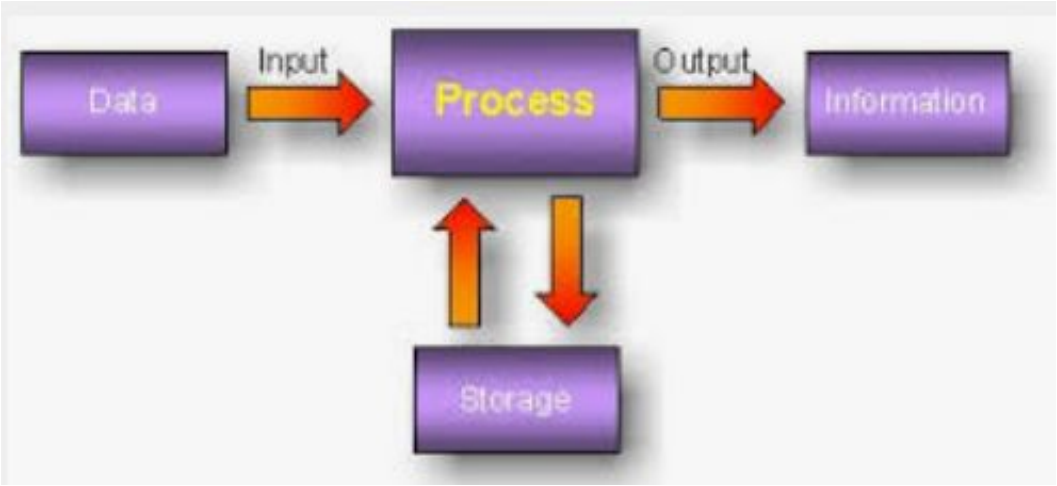
Information Processing

Information processing refers to the **systematic sequence of operations** performed on data to convert it into useful information. It is the **core activity of a computer system**

Steps of Information Processing

- **Input**
  - Collecting and entering data into the computer.
  - Devices: Keyboard, Mouse, Scanner, Microphone.
- **Storage**
  - Data is stored temporarily or permanently for processing.
  - **Primary storage**: RAM, Cache (fast, volatile).
  - **Secondary storage**: Hard disk, SSD, Optical disks (non-volatile).
- **Processing**
  - Actual manipulation of data into information.
  - Done by **CPU (Central Processing Unit)** using:
    - **Arithmetic operations**: Addition, subtraction, multiplication, division.
    - **Logical operations**: Comparisons, decision-making (>, <, =).

- **Output**
  - Processed results are presented in human-understandable form.
  - Devices: Monitor, Printer, Speakers.
- **Feedback** (Optional but important)
  - Output can be re-entered into the system for corrections, improvements, or further processing.
  - Example: After getting exam results (output), a teacher may reprocess data for averages (feedback → processing).



Characteristics of Good Information

For processed data (information) to be **useful**, it should have these qualities:

- **Accuracy** – Free from errors.
- **Timeliness** – Available when needed.
- **Relevance** – Related to the decision-making context.
- **Completeness** – All necessary details included.
- **Conciseness** – Presented clearly and simply.
- **Cost-effectiveness** – Benefits of information should outweigh its cost.

Real-life Example

Let's take a **Banking Example**:

- **Data**: `10000, 2000, 5000`
- **Processing**: Bank software calculates deposits, withdrawals, and balance.
- **Information**: "Your account balance is ₹13,000."
- **Output**: Displayed on ATM screen or SMS.
- **Feedback**: Customer may withdraw again, leading to reprocessing.

What is a Digital Computer?

- A **digital computer** is a type of computer that processes data in the form of **digits (binary: 0 and 1)**.
- It uses **electronic circuits** to represent and manipulate these binary numbers.
- Digital computers are the most common type today – e.g., PCs, laptops, smartphones, servers.

Characteristics of a Digital Computer

- **Binary System**: Works on 0s and 1s (ON = 1, OFF = 0).
- **Programmable**: Can execute a set of instructions (software).
- **Versatile**: Can perform multiple tasks (mathematical, logical, multimedia, simulations).
- **Fast and Accurate**: Millions of operations per second with high precision.
- **Memory-based**: Stores both data and instructions.
- **Automatic**: Once started, it works automatically until the task is complete.

Functional Units of a Digital Computer

A digital computer has **five major components**:

(a) Input Unit

- Accepts data and instructions from the user.
- Converts them into machine-readable form (binary).
- Devices: Keyboard, Mouse, Scanner, Microphone.

(b) Central Processing Unit (CPU)

The **“brain” of the computer**. It controls and processes everything.
It has **three sub-units**:

- **Arithmetic Logic Unit (ALU)**
    - Performs all arithmetic operations: +, −, ×, ÷.
    - Performs logic operations: comparisons (>, <, =) and decision-making.
- **Control Unit (CU)**
    - Directs and coordinates the flow of data.
    - Acts like a traffic controller: tells input, ALU, memory, and output what to do.

- **Registers**
    - Small high-speed storage inside CPU.
    - Holds intermediate results, instructions, or addresses temporarily.

      Memory / Storage Unit:

      Stores data, instructions, and results.

      Two types:

      Primary Memory (RAM, ROM, Cache) – fast, volatile.

      Secondary Memory (Hard Disk, SSD, CDs, Pen drives) – permanent.
- Output Unit

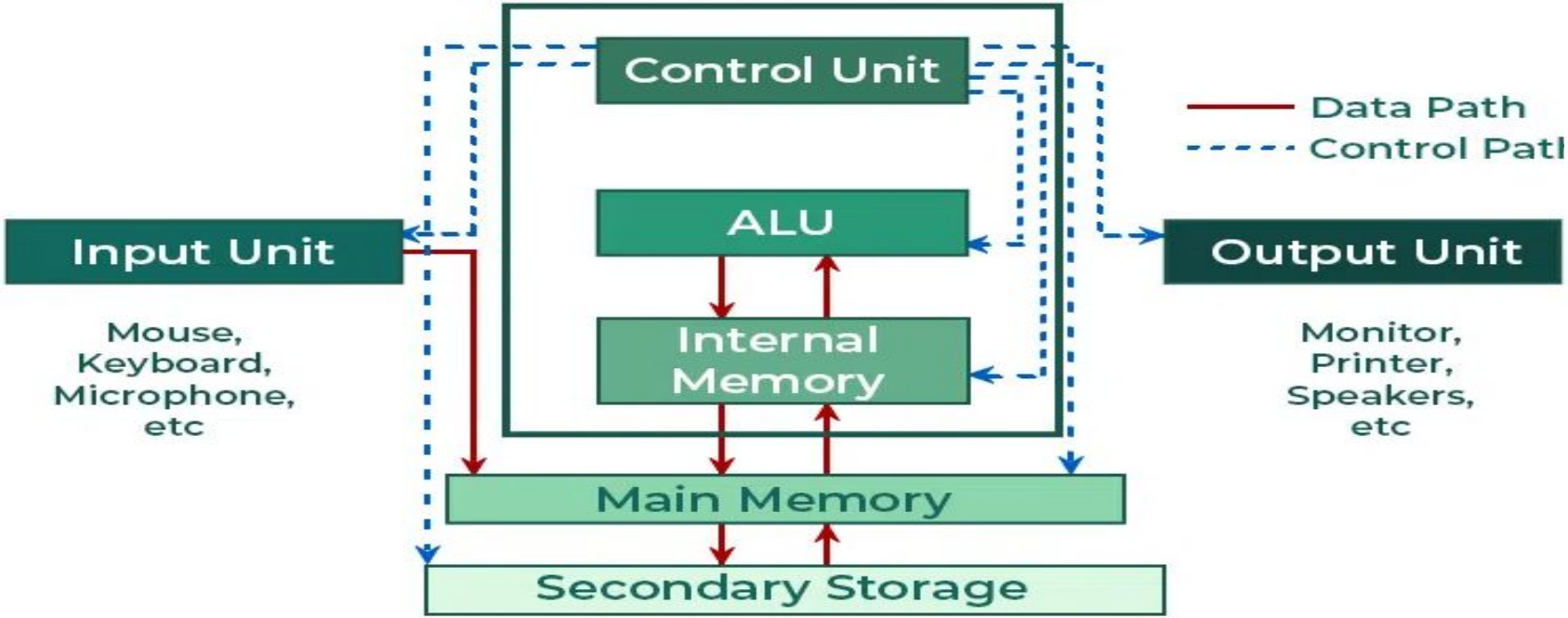
  Converts processed data (information) into human-understandable form.
  Devices: Monitor (visual), Printer (hardcopy), Speaker (audio).
- Communication Unit (optional)

  In modern computers, networking components allow computers to communicate with other systems.
  Devices: Network cards, Wi-Fi modules, Modems.

Working of a Digital Computer (Step by Step)

The working follows the **Information Processing Cycle**:

1. **Input Stage**
    - User gives raw data and instructions (e.g., entering marks through keyboard).
2. **Storage Stage**
    - Data is stored temporarily in **RAM** or permanently in **secondary storage**.
3. **Processing Stage**
    - **Control Unit** fetches instructions from memory.
    - **ALU** performs required operations.
    - Example: Calculating the student's average marks.
4. **Output Stage**
    - Processed results are displayed or printed in human-readable form.
    - Example: "Student's average = 85%".
5. **Feedback & Reprocessing**
    - If correction is needed, output can be fed back into input for reprocessing.



Real-life Example of Working

👉 Example: ATM Machine

- **Input**: User enters PIN and withdrawal amount.
- **Processing**: CPU checks PIN validity, account balance, and updates records.
- **Storage**: Transaction data stored in bank servers.
- **Output**: Cash dispensed, receipt printed, and message displayed.

Introduction to Computer Hardware and Software

A computer system is made up of **two main components**:

1. **Hardware** – The physical, tangible parts of the computer.
2. **Software** – The programs and instructions that tell hardware what to do.

👉 **Analogy:**

- Hardware = **Human Body (organs, bones, muscles).**
- Software = **Human Mind (thoughts, instructions).**
- Without software, hardware is just a useless box. Without hardware, software cannot run.

Computer Hardware

Definition : Hardware refers to the **physical components** of a computer that you can see and touch.

Types of Hardware

1. **Input Devices**
    - Allow users to enter data and instructions.
    - Examples: Keyboard, Mouse, Scanner, Microphone, Webcam.

2. **Output Devices**
    - Display or present processed information.
    - Examples: Monitor, Printer, Speakers, Projector.

Storage Devices

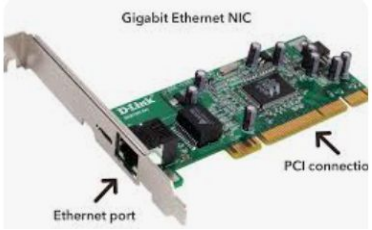
- Used to store data permanently or temporarily.
- Examples: Hard Disk, SSD, Pen Drive, CD/DVD.

Processing Devices

- Mainly the **CPU** (Central Processing Unit), which includes:
    - **ALU (Arithmetic Logic Unit):** Performs calculations.
    - **CU (Control Unit):** Directs operations.
    - **Registers & Cache:** Very fast temporary storage.

Communication Devices

- Help computers communicate with other computers.
- Examples: Network Card, Wi-Fi Adapter, Modem, Bluetooth device.



What is an Algorithm?

An **algorithm** is a **step-by-step set of instructions** to solve a problem or do a task.

- It is like a **recipe** in cooking:
    - A recipe tells you **what to do first, next, and last**.
    - Similarly, an algorithm tells the computer (or a person) the exact steps to follow to reach the solution.

Key Points

- It is **not a program** but a **plan** for solving a problem.
- It should be **clear and unambiguous** (no confusion).
- It must **end after a finite number of steps**.
- It can be written in **English, pseudocode, or flowcharts**.
    -

Example 1 (Real-life)

Algorithm to make tea

1. Boil water.
2. Add tea leaves.
3. Add milk and sugar.
4. Stir well.
5. Serve hot.

This is an algorithm for tea-making!

Example 2 (Computer-related)

Algorithm to add two numbers

1. Start.
2. Take input of two numbers (A, B).
3. Add A and B, store result in SUM.
4. Display SUM.
5. Stop.

Properties of a Good Algorithm

1. **Clear** – Steps are well defined.
2. **Finite** – Must end after some steps.
3. **Effective** – Each step must be doable.
4. **Input/Output** – Should take inputs and give correct outputs.

Example 2 (Computer-related)

Algorithm to add two numbers

1. Start.
2. Take input of two numbers (A, B).
3. Add A and B, store result in SUM.
4. Display SUM.
5. Stop.

Properties of a Good Algorithm

1. **Clear** – Steps are well defined.
2. **Finite** – Must end after some steps.
3. **Effective** – Each step must be doable.
4. **Input/Output** – Should take inputs and give correct outputs.

What is a Flowchart?

👉 A **flowchart** is a **diagram** that shows the **steps of a process (or algorithm)** using different **shapes and arrows**.

- It is like a **map of a program or task**.
- Shapes represent steps, and arrows show the **flow of work** (what happens first, next, last).

Why use a Flowchart?

- Makes it **easy to understand** the logic.
- Helps in **visualizing algorithms**.
- Useful in planning before writing actual code.

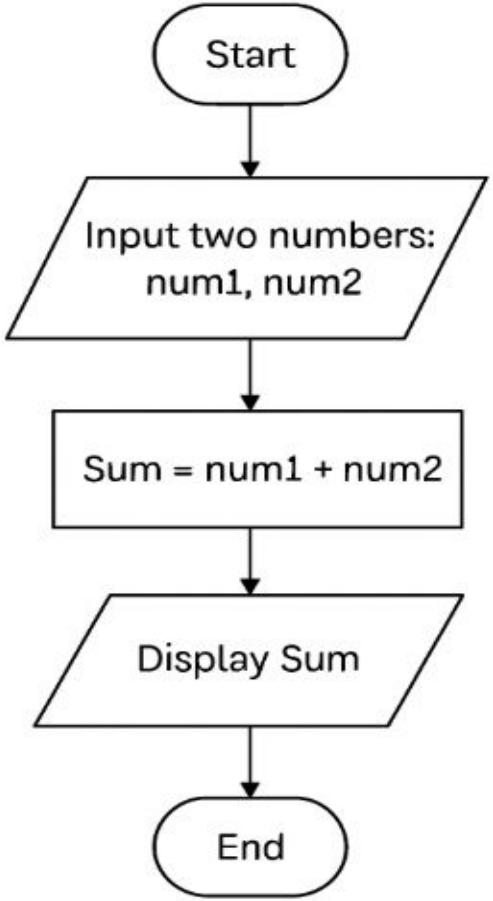
Basic Flowchart Symbols

Symbol	Meaning	Example
Oval 	Start / End	“Start”, “Stop”
⬜ **Rectangle**	Process / Action	“Add two numbers”
🔷 **Diamond**	Decision (Yes/No)	“Is A > B?”
Parallelogram 	Input / Output	“Enter number”, “Display result”

Algorithm to add two numbers

1. Start.
2. Take input of two numbers (num1, num2).
3. Add num1 and num2, store result in SUM.
4. Display SUM.
5. Stop.

Flowchart to add two numbers



Algorithm vs Flowchart

Aspect	Algorithm	Flowchart
Definition	Step-by-step **written instructions** to solve a problem.	A **diagram** that shows steps of a process using shapes and arrows.
Form	Written in plain language, pseudocode, or numbered steps.	Visual (graphical representation).
Understanding	Easy to write, but harder to visualize flow.	Easy to understand logic at a glance.
Symbols Used	No fixed symbols, just text.	Uses standard symbols (Oval, Rectangle, Diamond, etc.).
Example	1. Start → 2. Input A, B → 3. Add A+B → 4. Display result → 5. End	Shapes with arrows showing Start → Input → Process → Output → End
Use Case	Best for planning in words before coding.	Best for visualizing and presenting process flow.

Questions for Algorithm and Flowchart Practice

1. **Add Two Numbers :** Write an algorithm and draw a flowchart to input two numbers and display their sum.
2. **Find the Larger of Two Numbers :** Create an algorithm and flowchart to input two numbers and print the larger one.
3. **Check Even or Odd :** Write an algorithm and flowchart to check if a given number is even or odd.
4. **Find Factorial of a Number:** Create an algorithm and flowchart to calculate factorial of a given number (e.g., 5! = 120).
5. **Calculate Simple Interest :** Write an algorithm and flowchart to compute Simple Interest = (P × R × T) / 100.
6. **Check Positive, Negative, or Zero :** Create an algorithm and flowchart to check if a number is positive, negative, or zero.
7. **Find the Sum of First N Natural Numbers :** Write an algorithm and flowchart to calculate the sum of the first N natural numbers.
8. **Temperature Conversion :** Create an algorithm and flowchart to convert temperature from Celsius to Fahrenheit.
    - Formula: F = (C × 9/5) + 32.
9. **Find the Smallest of Three Numbers :** Write an algorithm and flowchart to input three numbers and display the smallest.
10. **Check Leap Year :** Create an algorithm and flowchart to check whether a given year is a leap year or not.
    - (Year is leap year if divisible by 400, or divisible by 4 but not 100).

Basic structure of a C program

- A **C program** is made up of different sections written in a specific order.
- Each section has a role: some are **optional**, some are **mandatory**.
- The **basic structure** makes sure the compiler understands and executes the program correctly.